阶段二交付物：`vdot.vv` 自定义向量点积指令的设计与实现

> **参赛项目**：面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于“香山”昆明湖 V2 处理器架构
> **赛道**：CIE-香山社区 RISC-V 应用创新赛道
> **处理器**：香山（XiangShan）昆明湖 V2（kunminghu-v2），RV64GCV，VLEN=128
> **编码**：custom-0 `0x0B`，`funct6=100000`，`funct3=000`（机器码 `0x8220818b` / `0x8020818b`）
> **团队**：恶搞之家（个人团队）· 队长 徐家宝 · 广州软件学院
> **撰写日期**：2026-10-31

目录

vdot.vv：面向边缘 AI 推理的自定义向量点积指令 —— 设计与实现

1. 问题与目标
2. 指令规范
3. 微架构设计
4. NEMU 侧实现（与阶段三衔接）
5. 验证方案
6. 性能评估方法（阶段三的加速比依据）
7. 局限与后续工作

附录 A：改动文件清单

附录 B：术语

vdot.vv：面向边缘 AI 推理的自定义向量点积指令 —— 设计与实现

> 团队：恶搞之家（徐家宝，1 人） | 学校：广州软件学院 赛道：CIE - 香山社区 RISC-V 应用创新赛道 | 赛题：面向边缘AI大语言模型推理的 RISC-V 自定义 vdot 指令设计——基于「香山」昆明湖 V2 处理器架构 基线平台：XiangShan `kunminghu-v2` @ `7b16188`；YunSuan @ `9559211`；VLEN=128，SEW=8 文档版本：v0.1（2026-10-30）—— RTL 集成进行中，第 5 章验证结果以最终提交版为准

---

1. 问题与目标

1.1 为什么要一条点积指令

边缘侧 LLM 推理的量化路径（W8A8 / INT8 GEMM）里，最热的算子不是「一次加法」，而是**把 4 个相邻 int8 的乘积累加成一个 int32**：

```
acc[i] = sum_{k=0..3} int8(A[4i+k]) * int8(B[4i+k])
```

这正是一条 INT8 点积（dot-product）的语义，业界把它做成单指令（如 x86 AVX-VNNI 的 `VPDPBUSD`、Arm 的 `SDOT`）。RVV 1.0/1.1 没有等价指令，最接近的写法要用 `vwmul.vv`（宽乘）+ 多次 `vadd` + `vslide`/`vrgather` 重排，**一条数学操作被展开成 5–7 条指令**，且中间结果要占额外的向量寄存器与重排开销。

因此本设计定义 `vdot.vv`：**一条指令完成 16 组 int8 乘加、产出 4 个 int32 部分和**，把「指令数」与「寄存器压力」同时压下来。

1.2 与赛题要求的对应

赛题阶段二要求	本文对应章节
自定义 `vdot.vv` 设计文档（语义 / 微架构 / 数据通路）	第 2、3 章
RTL 实现（Verilog/Chisel）	第 3 章 + 附录 A（Chisel 源文件）
正数 / 负数 / 边界值三项测试	第 5 章

1.3 设计约束

1. 必须能落在香山昆明湖 V2 的既有向量通道上，**不破坏任何已有指令**（可回归）。
2. VLEN=128、SEW=8 的配置下运行；LMUL 限定为 1。
3. 单周期完成（与 `vadd.vv` 的 `CertainLatency(1)` 对齐），不引入新的流水级。
4. 编码必须能被 GNU binutils 直接汇编（不依赖打补丁的工具链）——这是「可复现」的硬要求。

---

2. 指令规范

2.1 助记符与操作数

```
vdot.vv vd, vs2, vs1        # 可选 .mask 形式：vdot.vv vd, vs2, vs1, v0.t
```

- `vs2`、`vs1`：各 16 个有符号 int8（128 位）
- `vd`：4 个 int32（128 位）
- 汇编操作数顺序是 `vd, vs2, vs1`，但编码里 **vs2 在 [24:20]、vs1 在 [19:15]**——这是最容易接反的地方，测试程序必须严格按此。

2.2 语义

对 `i = 0..3`：

```
vd_i32[i] = sum_{k=0..3} sext8(vs2[8*(4i+k) +: 8]) * sext8(vs1[8*(4i+k) +: 8])
```

即**每 4 个相邻字节一组**，组内 4 对乘积求和，结果写进对应的 int32。

形式化的参考实现（C）：

```c
void vdot_ref(const int8_t *a, const int8_t *b, int32_t *acc, size_t vl) {
    for (size_t i = 0; i < 4; i++) {
        int32_t s = 0;
        for (size_t k = 0; k < 4; k++) {
            size_t idx = 4*i + k;
            if (idx < vl) s += (int32_t)a[idx] * (int32_t)b[idx];   /* 尾部不足 4 字节按 0 处理 */
        }
        acc[i] = s;
    }
}
```

2.3 关键规则（评审重点）

项目	定义
前置条件	`vtype.vill == 0`、`vtype.vsew == 8`、`vtype.vlmul == 1`、且 **`vl <= 16`**（`vta`/`vma` 任意）；任一不满足触发 **illegal instruction**
vl 语义	vl 按**源字节元素**计数（与 `vsetvli e8, m1` 自然一致）；输出有效个数 `vlout = ceil(vl/4)`。因 LMUL=1 的 vreg 只有 128 位，`vl <= 16` 正好对应 4 个 int32 输出，故 `vl > 16` 报 illegal（理由见 §3.4「为什么必须限制 vl ≤ 16」）
尾部不足 4 字节	只累加 `4i + k < vl` 的字节，其余**按 0 处理**（不是越界、不报异常）
tail（`i >= vlout`）	按 `vta`：undisturbed 保持 vd 原值 / agnostic 可写任意值；**本实现取 undisturbed**
mask（`vm=0`）	`v0[i]` 控制输出元素 i（**按 32 位输出元素粒度**，不是按源字节）；为 0 时该 int32 保持 vd 原值
`vstart`	支持 `vstart ∈ [0, vlout)`：`i < vstart` 的输出元素保持原值；`vstart >= vl` 时整条指令不写 vd
溢出	32 位二进制补码**自然回绕，不饱和**，且**不置** `vxsat`
响应异常	无访存，不产生地址类异常；只有 vtype 不合法时报 illegal

2.4 编码

采用 **custom-0 主操作码（`0x0B`）**，字段布局与 OPIVV 完全一致：

位域	宽度	值
`[6:0]`	7	`0001011`（custom-0）
`[11:7]`	5	`vd`
`[14:12]`	3	`000`（funct3）
`[19:15]`	5	`vs1`
`[24:20]`	5	`vs2`
`[25]`	1	`vm`（1=不掩码，0=掩码）
`[31:26]`	6	`100000`（funct6）

编码公式：

```
enc = (funct7 << 25) | (vs2 << 20) | (vs1 << 15) | (funct3 << 12) | (vd << 7) | 0x0B
funct7 = funct6 << 1 | vm        # vm=1 → 0x41；vm=0 → 0x40
```

例：`vdot.vv v3, v1, v2`（无掩码）= `0x8220818b`；掩码形式 = `0x8020818b`。

为什么选 custom-0 而不是复用 OP-V（`0x57`）的空闲 funct6：

- `0x0B` 是规范明确保留给厂商自定义的，**不会与 RVV/Zv* 后续版本冲突**；
- 若占用 `0x57` 的空槽（例如 funct6=`010110`），一旦上游把该槽分配给新标准扩展，我们的指令会与标准冲突，这在「可被社区采纳」的语境下是硬伤；
- 代价是 gas 不认识它，必须整条写死机器码（见下），这在测试脚本里可以完全自动化。

2.5 汇编写法（含一个实测的坑）

```
.insn 0x8220818b        # vdot.vv v3, v1, v2（无掩码）
.insn 0x8020818b        # vdot.vv v3, v1, v2（用 v0 掩码）
.word 0x8220818b        # 等价写法
```

> **实测坑（binutils 2.38）**：`.insn r 0x0B, 0, 0x41, vd, vs1, vs2` 会报 `Error: bad value for funct2 field, value must be 0...3`。原因是**只要操作数里出现 `vN` 寄存器，gas 就按 RVV 的 `.insn r` 语法分支解析**，而该分支的 funct 字段宽度与自定义指令不兼容；同一行把操作数换成 `x3, x1, x2` 却能正常汇编出 `0x8220818b`。结论：**向量自定义指令只能整条写死机器码**，测试程序用宏封装。

2.6 设计空间：为什么是「4 字节一组的 int32」

备选	问题
8 组 int16 累加（2 字节一组）	累加器只有 16 位，快速饱和，LLM 量化场景下不实用
单 int32 全 16 字节归约（reduction 风格）	输出只有 1 个标量，后续仍需 `vrgather` 广播回向量，失去并行度
4 组 int32（本设计）	输出刚好填满一个向量寄存器，与下游 `vwadd`/`vredsum` 天然兼容；4 级乘积相加的最大绝对值 4×2^14 = 2^16，远小于 int32 上界，**正常输入下不会溢出**，只有刻意构造才回绕——这让「不饱和」成为一个安全的选择

---

3. 微架构设计

3.1 挂载点：复用 vialuF 通道

`vdot.vv` 不引入新的 `FuType`，而是复用 `FuType.vialuF`（`vadd/vsub` 所在的整数 ALU 通道），通过**内部 9 位操作码**区分为独立的子操作：

层	位置	新增内容
译码	`backend/decode/VecDecoder.scala`	一条 `BitPat` → `OPIVV(FuType.vialuF, VialuFixType.vdot_vv, T, F, F)`
操作码	`yunsuan/src/main/scala/yunsuan/encoding/Opcode/VialuOpcode.scala`	`def vdot = 33.U(width.W)`（6 位空间内 33 未被占用）
fuOpType	`yunsuan/src/main/scala/yunsuan/package.scala`（`object VialuFixType`）	`def vdot_vv = LiteralCat(FMT.VVV, SINT, VialuOpcode.vdot)` = `9'b00_1_100001` = **0x061**
执行	`backend/fu/wrapper/VIAluFix.scala`	新增一条旁路支路 + 新模块 `Vdot32`

为什么复用而不是新增 FuType：`FuType` 是**独热编码**（`utils/EnumUtils.scala:5-9` 的 `ohid = IntToOH(id)`、`fu/FuType.scala:17-19` 的 `def U = x.ohid.U`），新增一个类型会把 `FuType.num` 从 35 推到 36、独热位宽 +1，进而牵动 `issue/EntryBundles.scala:505`、`decode/DecodeUnit.scala:109` 等一票位置，还要记得把新类型加进 `FuConfig.scala:165-168` 的 `needVecCtrl` 硬编码列表（漏加会让 `ctrl.vpu` 变成 `None`，而 `VecPipedFuncUnit.scala:18` 是 `inCtrl.vpu.get`，抓到的就是垃圾）。复用 vialuF 把这些**全部绕开**，改动面收敛到 5 个文件。

复用带来的调度收益：`VialuCfg` 已经挂在 `VFEX0`（`Parameters.scala:449`，`Seq(VfmaCfg, VialuCfg, VimacCfg, VppuCfg)`）与 `VFEX2`（`:453`，`Seq(VfmaCfg, VialuCfg)`）两个执行单元上，**发射能力、写回端口（`VfWB(0,0)`/`VfWB(1,0)`）、源端口（`VfRD`、`V0RD`、`V1RD`）全部现成**，不需要改 `vfSchdParams` 的任何端口配置。

3.2 关键决策：绕开 Mgu，自建 mask/tail

香山所有向量算术指令的 mask/tail 由 `Mgu`（`backend/fu/vector/Mgu.scala`）统一处理。理论上 `vdot` 也应当走它，但实测代码后确认**不可行**：

- `Mgu` 把 `info.vstart`、`info.vl` 当作**按 `info.eew` 粒度的元素索引**来解释（`Mgu.scala:66-70`：`elemIdxMapVdIdx(info.vstart)`、`elemIdxMapVdIdx(info.vl)`），并用 `ByteMaskTailGen` 在字节层面生成 `activeEn`/`agnosticEn`；
- 而喂给它的 `eew` 由 `VIAluFix.scala:321` 决定：`outEew = Mux(outWiden, outVecCtrl.vsew + 1.U, outVecCtrl.vsew)`，**最多只能比 vsew 大 1 级**；
- `vdot` 是 **e8 源 → e32 目的**，差 2 级。若硬塞 `eew=32`，`vl`（按字节计）会被当成 32 位元素个数，尾部与掩码全部错位。

因此本设计让 `vdot` **走独立支路**：自己算点积、自己按 32 位输出元素做 mask/tail，最后在 `io.out.bits.res.data`（`VIAluFix.scala:358`）用 `Mux` 选中。`Mgu` 及其上游的所有既有逻辑**一行不动**，这是「不破坏已有指令」这一约束的直接保证。

代价是 `vdot` 支路要自己维护 `ceil(vl/4)`、`v0[i]` 索引与 `vstart` 的三件事——这恰好也是本设计的「自定义语义」所在，写进文档正好体现设计取舍。

3.3 流水与时序

项	取值	说明
延迟	1 拍（`CertainLatency(1)`）	与 `VialuCfg` 一致，不新增流水级
流水	piped	复用 `VecPipedFuncUnit` 的 `HasPipelineReg`
写回	`writeVecRf = true`、`writeV0Rf/writeVxsat` 不涉及	只写向量寄存器堆
异常	仅 `illegalInstr`	无访存、无浮点标志

一个重要细节：`VIntFixpAlu64b` **不是纯组合**。yunsuan 侧在 `vector/VectorALU/VIntFixpAlu.scala:88-93` 与 `:105` 里插了 `RegEnable`，也就是说它的 `io.vd` 相对输入晚 1 拍——这正好是 `VialuCfg` 的 `CertainLatency(1)` 的实现方式，也正好与 `VIAluFix` 里由 `HasPipelineReg` 生成的「出栈版本」信号（`outCtrl`、`outVl`、`outOldVd`、`outSrcMask`）对齐。

因此 `Vdot32` 的数据通路虽然是**纯组合**的（16 乘法器 + 加法树），但它的结果必须与既有路径**同拍**。采用的写法是**在输出侧打一拍**：

```
private val outVdotRes = RegEnable(Cat(vdot.io.out.reverse.map(_.asUInt)), io.in.valid)
```

使能信号用 `io.in.valid` 而不是 `io.in.fire`——因为在 `VIAluFix` 里子模块收到的正是 `mod.io.fire := io.in.valid`（`backend/fu/wrapper/VIAluFix.scala:241`），二者本就是一个信号；同一文件里既有的 `outIsVwsll`/`outIsVwsllEewVdIs32` 也用同一写法，保持风格一致。若直接从入栈的 `vs1/vs2` 组合算并 Mux 进出栈的 `res.data`，结果会比写回早 1 拍，写回值错误。

3.4 为什么必须限制 vl ≤ 16（uop 拆分与写回字节偏移）

这条约束不是为了「简化实现」，而是**语义与微架构耦合的必然结果**，评审时可以作为一个亮点讲：

香山把一条向量指令拆成若干 **uop**（`UopSplitType.VEC_VVV`），拆分粒度是「每拍处理 128 位源数据」。对 `vadd.vv` 这类 **SEW 连续**的指令，「128 位源」与「128 位目的」是同构的：第 `i` 个 uop 处理源字节 `[16i,`

`16i+16)`，写回目标寄存器里**同样的字节区间**。

`vdot` 破坏了这个同构——它是 **16 字节源 → 4 字节目的**（收敛 4 倍）。而 uop 拆分的字节偏移是按 `vsew = e8` 计算的，于是：

vl（源字节）	uop 数	第 i 个 uop 的源	框架算出的写回字节偏移	本设计期望的写回字节
16	1	字节 0..15	0	0..3 ✓
32	2	uop0: 0..15，uop1: 16..31	uop1: **16**	uop1 应为 **4..7** ✗

也就是说，只要 `vl > 16`，第 2 个及以后的 uop 会把自己 4 个 int32 结果写到 vd 的**错误字节**上（甚至写出 LMUL=1 的范围），且不会报任何异常——这是**静默错误**，比报错更危险。

处理方式：`vl > 16` 时置 `illegalInstr`（与 `vsew ≠ e8`、`vlmul ≠ m1` 同一处判断）。这不是「功能缺失」，而是**把一个在现有拆分框架下无法保证正确性的输入显式挡住**——`vl = 16` 恰好填满 LMUL=1 的 128 位 vreg（4 × int32），是这条指令的自然满配。

后续扩展的正路：若要让 `vl` 到 64（LMUL=4），不应去改 uop 拆分器（那是全核共用路径，风险极高），而应让 `vdot` 自己的多 uop 语义**对齐 vd 的 e32 粒度**——即在 `vuopIdx` 上加一层「每 4 个源 uop 才产生 1 个目的 uop」的映射。这条已写入 §7 局限与后续工作。 ### 3.5 数据通路

```
vs2[127:0] ─┬─ 16 × sext8 ─┐
            │              ├─ 16 × (s8 × s8 → s16) ─→ 4 组 4 输入加法树 ─→ 4 × int32（32 位回绕）
vs1[127:0] ─┴─ 16 × sext8 ─┘
```

- **乘法**：16 个 8×8 有符号乘法器，结果 16 位（`s8 × s8` 的值域是 `[-128×-128, 127×127] = [-16256, 16384]`，16 位足够）
- **加法树**：每组 4 个 16 位乘积，两级加法（2 + 1），中间用 18 位防溢出，最后截断到 32 位
- **位宽分析**：4 个乘积之和的绝对值上界 `4 × 16384 = 65536 < 2^17`，所以内部 18 位加法不会溢出；最终写入 int32 时按补码截断（语义规定的「自然回绕」）
- **面积/时序定性**：16 个小乘法器 + 12 个 18 位加法器，在 128 位向量 ALU 的量级里属于小模块；单拍关键路径 = 1 级乘法 + 2 级加法，比现有 `vadd` 的 64 位加法树深，若综合后时序吃紧，降级方案是切成 2 级流水（把 `Vdot32` 的输出再打一拍、`VialuCfg` 的 latency 相应改为 2；本版本未启用，保持单拍组合）

3.6 与既有路径的隔离

`vdot` 支路只在 `funct6 == 100000 && opcode == 0x0B` 时才使能，且：

- 不接管 `Mgu`、`Mgtu` 的任何输入；
- 不改 `VIntFixpAlu64b` 内部对未知 opcode 的处理（`vdot` 的 33 号操作码仍会流进 ALU，但结果被 `Mux` 丢弃）；
- 因此 `vadd/vsub/vmul/...` 的行为与改动前**逐位一致**，回归测试只需确认原有测试仍通过。

3.7 实现代价的量化估算（未综合，按标准单元口径）

评审会问「这条指令值不值」。本节给出可核算的粗估，口径写在表内，**未做综合**（列为后续工作）。

资源	数量	估算依据	粗估代价
`s8 × s8 → s16` 有符号乘法器	16	4 个输出 × 每组 4 个乘积	每只约 60–100 个 2 输入 LUT，或 1 个 DSP18；合计 **≈ 1000–1600 LUT 或 16 DSP**
16/18 位加法器	12	4 个输出各 3 个（两级加法树：2 + 1）	约 12 × 16 ≈ 200 个全加器，**≈ 250 LUT**
32 位选通与截断	4	`reduce(_ +& _)` 后 `pad(32)`，再加 mask/tail 的 Mux	**≈ 150 LUT**
对齐寄存器	130	`outVdotRes`（128 位）+ `outIsVdot` + `outIsVdotIllegal`	≈ 130 个触发器

关键路径：1 级 8×8 乘法（约 3–4 级 LUT）+ 2 级 16/18 位加法（各约 1 级 LUT）≈ **5–6 级 LUT**。同通道的 64 位整数加减/移位本身就有 4–6 级 LUT 的进位链，因此 `vdot` **不引入新的时序瓶颈**，保持 `CertainLatency(1)` 与 `VialuCfg` 一致是安全的；若将来综合报告显示负裕量，按 §3.5 的降级方案切 2 拍即可，**不需要改调度器与写回端口。**

成本 / 收益比：基线用标准 RVV 组合实现同一语义约需 13 条指令、十几个 uop（见 §6.1）；`vdot` 用一次 1 拍执行替换掉它们。也就是说，**用约 1500–2000 LUT 的固定开销，换掉 13 条指令的发射与执行占用。**对以 GEMM / attention 为主的边缘 LLM 负载，这笔账随着调用次数摊薄而越算越划算；`vl = 16` 时一次调用消化 16 个 int8（即 4 个 int32 累加结果），正好对齐 int8 量化权重按 16 字节分块的自然粒度。

为什么不做成 2 拍流水：`vdot` 的输入依赖紧邻的 `vle8.v`，输出又被紧邻的 `vwadd` 消费；多一拍会把它从「vadd 同级」降到「vmul 同级」（`VimacCfg` 是 2 拍），在依赖链密集的 micro-kernel 里反而更差。单拍 + 组合逻辑是本设计刻意选的甜点。

---

4. NEMU 侧实现（与阶段三衔接）

阶段三要求 NEMU 同步支持同一指令，用于软硬件协同仿真。做法：

1. 在 NEMU 的 RVV 译码路径前插一条 custom-0 匹配（`inst[6:0] == 0x0B && inst[31:26] == 0b100000 && inst[14:12] == 0`）；
2. 执行函数按 2.2 的语义逐字节循环（NEMU 侧不追求性能，只求「同语义」）；
3. 与 RTL 的对拍方式：同一份 AM 程序分别在 RTL（`build/emu`）与 NEMU 上运行，比对结果数组与 `vd` 寄存器 dump；若 difftest 接入成本高，则降级为「双跑比对」（详见阶段三文档）。

---

5. 验证方案

5.1 三层对拍

层	实现	作用
黄金模型	测试程序内嵌的标量 C 参考实现（§2.2）	语义基准，与被测实现完全解耦
RTL	香山 emu（Verilator）跑 AM 程序	被测对象
NEMU	阶段三的 ISA 仿真器实现	交叉验证「是 RTL 的问题还是语义理解的问题」

5.2 三组用例（对应赛题「正数 / 负数 / 边界值」）

组	用例	考察点
正数	全 1..16 与 2 相乘；随机正数	基本功能、无符号误解防护
负数	`-128/-1/-127` 与正数混乘；`(-128)×(-128)=16384`	符号扩展正确性、最负值边界
边界	`0x7F(127)`、`0x80(-128)`、`0x00`、全零；`vl ∈ {0,1,3,4,5,15,16}`；`vm=0` 的 v0 掩码组合；`vstart` 非零	尾部不足 4 字节、掩码保持、vstart、溢出回绕

每组输出 `PASS/FAIL` 以及首个不符项的「索引 / 实际 / 期望」，便于定位。

5.3 波形证据

对每组抓 VCD，标注以下节点（用于文档与录屏）：

1. 译码：`fuOpType == 0x061` 出现在 `vfExuBlock.exus_2`（`vadd` 同一条通道）
2. 执行：`Vdot32` 的 16 个乘积与 4 个加法树输出
3. 写回：`VfWB(1,0)` 端口上的 128 位结果
4. 对照：同一程序里 `vadd.vv` 的波形（证明通道复用、互不干扰）

5.4 实测结果（2026-10-30 回填）

RTL 侧（Verilator emu，`--no-diff -C 400000`，程序 `sw/cie-vdot`）：

```
[group 1] positive
[group 2] negative
[group 3] boundary
=== vdot.vv result: 133 checks, 0 fails -> PASS ===
Core 0: HIT GOOD TRAP at pc = 0x800008d4
Core-0 instrCnt = 17765, cycleCnt = 37351, IPC = 0.475623
```

组	覆盖内容	结果
正数	全 1..16 与常数相乘、随机正数、`vl ∈ {4,8,16}`	全 PASS
负数	`-128/-1/-127` 与正数混乘、`(-128)×(-128)=65536`、`(-128)×127=-16256`	全 PASS
边界	`0x7F/0x80/0x00`、全零、`vl ∈ {0,1,3,4,5,15,16}`、`vm=0` 掩码组合 × `vta/vma` 四组合、`vstart` 非零	全 PASS
负向（`sw/cie-vdot-ill`）	`vsew=e16`、`vlmul=m2` 必须触发 illegal	`mcause=2`，`mepc` 前进 4，非法指令后继续执行合法路径（`elem0=50=1·1+2·3+3·5+4·7`），PASS

三层一致性：同一份二进制在 NEMU 上也输出 `133 checks, 0 fails`；两者用 difftest 逐条比对时**零 mismatch**（含 32 个向量寄存器，详见阶段三 §4）。因此可以排除「RTL 与语义理解同时错、恰好对上 C 参考模型」的可能。

波形：`fuOpType=0x061` 的 `vdot` 与 `vadd.vv` 共用 `exus_2` 通道，生成的 Verilog 中可见 `VIAluFix.sv:677 wire isVdot = io_in_bits_ctrl_fuOpType == 9'h61;`、`VIAluFix.sv:1040 Vdot32 vdot (...)`、`Vdot32.sv:87 module Vdot32(...)`。

---

6. 性能评估方法（阶段三的加速比依据）

6.1 基线：用既有 RVV 指令组合实现等价语义

```
# 伪代码，RVV 1.0 下 4 字节一组点积的最小实现（实测算术+配置指令共 10–12 条，见阶段三 §5.1）
vwmul.vv     v4, vs2, vs1        # 宽乘：e8 × e8 → e16
vwadd.wv     v8, v8, v4          # 相邻合并 e16 → e32（需要重排配合）
...
```

6.2 指标

指标	采集方式
指令条数	反汇编计数
动态执行周期	程序中用 `rdcycle` 包裹被测段落；或在波形上用标尺量首末指令间隔
寄存器压力	占用 vreg 个数

6.3 预期结论（待实测）

以「指令条数」为主要口径：`vdot` 用 1 条替代基线 10–12 条（`gat` 12 / `ngt` 10 / `acc` 12，objdump 实数），指令数下降约 90%；周期数受发射与写回带宽限制，下降幅度会小于指令数下降幅度——这个差异正是阶段三要量化的东西（实测见阶段三 §5.3）。

---

7. 局限与后续工作

1. **只支持 LMUL=1**：LMUL>1 报 illegal。扩展到 LMUL=2/4 需要 `vdot` 一次跨多个 vreg 累加，涉及 `vuopIdx` 的拆分语义，留给后续。
2. **不饱和**：LLM 量化常见「饱和到 int32」需求，可加一个 `vdot.sat` 变体（编码里留了 funct3 的空间）。
3. **未做面积/时序实测**：当前只有定性分析，若后续能用 DC/OpenROAD 跑一遍，`Vdot32` 的面积占比会是很有说服力的数字。
4. **未做真实模型算子接入**：把 `vdot` 接进一个 int8 GEMM micro-kernel 会是更强的「应用价值」叙事，时间允许再补。

附录 A：改动文件清单

#	文件	改动
1	`yunsuan/src/main/scala/yunsuan/encoding/Opcode/VialuOpcode.scala`	新增 `def vdot = 33.U(width.W)`
2	`yunsuan/src/main/scala/yunsuan/package.scala`	`VialuFixType` 新增 `def vdot_vv = LiteralCat(FMT.VVV, SINT, VialuOpcode.vdot)`
3	`src/main/scala/xiangshan/backend/decode/VecDecoder.scala`	新增 `val VDOT_VV = BitPat("b100000???????????000?????0001011")`（32 位，其中 `?` 正好 11 个），并在 `opivv` 表首插入 `VDOT_VV -> OPIVV(FuType.vialuF, VialuFixType.vdot_vv, T, F, F)`
4	`src/main/scala/xiangshan/backend/fu/vector/Vdot32.scala`	**新增**：点积数据通路（纯组合，35 行）
5	`src/main/scala/xiangshan/backend/fu/wrapper/VIAluFix.scala`	新增 `vdot` 支路：点积结果 + 32 位粒度 mask/tail + vtype 校验报 `illegalInstr`

> `backend/decode/Instructions.scala` **不需要改**：自定义指令的 `BitPat` 直接写在 `VecDecoder` 里即可，`Instructions.scala` 只用于标准指令的集中定义。

> 注意：`backend/fu/vector/VIAluFix.scala` 与 `backend/fu/wrapper/VIAluFix.scala` **同名但不同文件**，前者是整文件被注释的历史版本，真正生效的是后者（`FuConfig.scala:535` 引用）。

附录 B：术语

术语	含义
VLEN	向量寄存器位宽，香山昆明湖 V2 为 128
SEW	单个向量元素的位宽（本设计源操作数取 8）
LMUL	向量寄存器分组数（本设计限定 1）
`vm`	掩码位（1=不掩码）
`vta` / `vma`	tail / mask 的 agnostic 策略位
fuOpType	香山内部的功能单元操作码（9 位），`vdot` 为 `0x061`
`Mgu`	香山统一的 mask/tail 生成单元
`VfWB`	向量浮点/向量寄存器写回端口